# Learning to Route

## An RL-Gated Policy for Hierarchical Speculative Decoding

viasd-rl

June 20, 2026

# Overview

**We introduce the first reinforcement-learned routing policy for hierarchical speculative decoding.**

Speculative decoding accelerates LLM inference; VIA-SD adds a slim intermediate verifier but routes between tiers with *hand-tuned confidence thresholds*.

We present **three key innovations**:

1. a **learned per-token gating policy** that recasts tier-routing as a sequential decision problem — a tiny $q$-free MLP replaces the fixed thresholds;
2. a **quality–cost reward** that balances matching the target model against expensive-call latency;
3. a **per-token RL formulation** with principled credit assignment (REINFORCE $\rightarrow$ GRPO $\rightarrow$ per-token advantage).

We challenge the assumption that hierarchical verification needs hand-tuned thresholds: a learned gate **Pareto-dominates** the fixed-threshold baseline, recovering *lossless* quality at a fraction of the expensive verifier calls.

# The bottleneck: LLM decoding is slow and sequential

- Each token **streams the entire model** from memory — inference is *memory-bandwidth bound*.
- **Speculative decoding (SD):** a cheap *drafter* guesses several tokens; the big *verifier* checks them in **one parallel pass**.
- Accepted guesses $\Rightarrow$ many tokens per expensive forward. **Lossless**, $\sim$**1.4**$\times$ wall-clock.
- The one lever: the **acceptance rate** — every rejected token forces a full-model step.

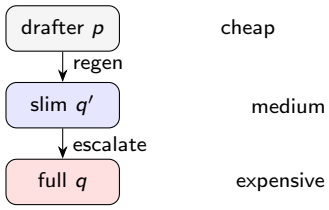
# VIA-SD improves on vanilla SD

**Vanilla SD** — binary per token:

- ACCEPT drafter, or REJECT → full model $q$.

**VIA-SD** — adds a **slim middle verifier** $q'$ ($q$ with ∼45% of layers skipped):

- ACCEPT — REGENERATE ($q'$) — ESCALATE ($q$).
- "Middle-zone" tokens handled *cheaply* by $q' \Rightarrow$ fewer full-model calls.



# . . . but it routes with hand-picked thresholds

## VIA-SD's routing rule

Two fixed confidence thresholds on $q'$: $\boxed{\alpha_1 = 0.5}$ $\boxed{\alpha_2 = 0.3}$

- **Global & static** — one cutoff for every token and context.
- **Brittle** — must be re-tuned per model and task.
- **Sub-optimal** — even fully swept, accuracy peaks at $\leq 0.50$ on our setup.

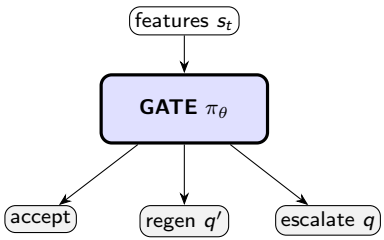
*Choosing a tier per token is a sequential, context-dependent decision — so* **learn** *it.*

# Innovation 1/3 — a learned per-token gating policy (MDP)

Recast tier-routing as a **Markov decision process**:

- **State** $s_t$: $q$-free features — drafter/$q'$ confidence, agreement, $\mathrm{KL}(p\|q')$, position.
- **Action** $a_t \in \{\text{accept, regenerate, escalate}\}$.
- **Policy** $\pi_\theta(a_t \mid s_t)$: a tiny MLP.

Key property: $q$ is consulted only to *score* at train time — **never** a gate input, so the gate is free to run at inference.



# Innovation 2/3 — a quality–cost reward

**Two ingredients per token:**

$$\begin{aligned} \text{quality:} \quad & \text{match}_t = \mathbf{1}\left[\hat{y}_t = \arg\max_v q(v \mid x_{<t})\right] \\ \text{cost:} \quad & \text{cost}_t = \left(t_{p_1} + t_{q'}/\gamma + \mathbf{1}[\text{ESCALATE}]\, t_q\right) / \, t_{q_1} \end{aligned}$$

**Reward** (cost in units of one full-model step):

trajectory: $R = \overline{\text{match}} - \lambda\, \overline{\text{cost}}$ $(+\, r_c \,\text{correct})$ per-token: $r_t = \text{match}_t - \lambda\, \text{cost}_t$

$\lambda$ is the **quality↔speed knob**; `match` is a *dense* proxy for the sparse true objective (final-answer correctness).

# Innovation 3/3 — per-token RL with credit assignment

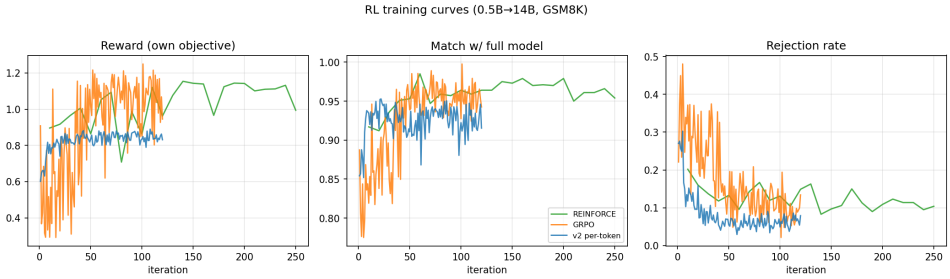
**Policy gradient** (all variants, $+\eta\,\mathcal{H}[\pi_\theta]$ entropy):

$$\nabla_\theta J = \mathbb{E}\Big[\sum_t A_t\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big]$$

They differ in the **advantage** $A_t$:

- **REINFORCE** (trajectory): $A = R - b$, EMA baseline $b \leftarrow (1-\tau)b + \tau R$.
- **GRPO** (group): $A_i = \dfrac{R_i - \mathrm{mean}_{\mathcal{G}} R}{\mathrm{std}_{\mathcal{G}} R}$, with PPO-clip $\rho_t = \frac{\pi_\theta}{\pi_{\theta_{\text{old}}}}$ and KL to $\pi_{\text{ref}}$.
- **per-token (v2):** $\hat{A}_t = \dfrac{r_t - \mathrm{mean}_{\text{batch}}\, r}{\mathrm{std}_{\text{batch}}\, r}$ — local credit, **lowest variance**.

# Training converges — per-token (v2) is cleanest



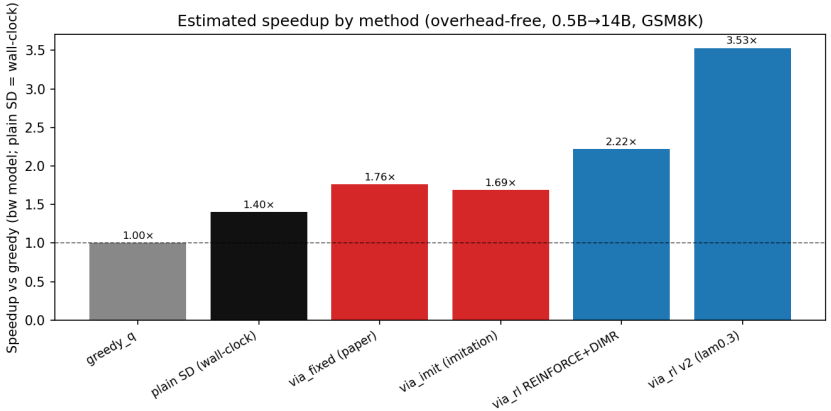
# Results: the learned gate dominates

| Method | GSM8K acc | q-calls/tok | speedup |
|---|---|---|---|
| greedy-$q$ (reference) | 0.887 | 1.000 | 1.00× |
| plain SD (standard, lossless) | 0.907 | 0.259 | 1.40×† |
| via_fixed (paper, tuned) | 0.40 | 0.31 | 1.76× |
| via_imit (imitation only) | 0.34 | 0.29 | 1.69× |
| via_rl REINFORCE | 0.880 | 0.243 | — |
| via_rl GRPO | 0.853 | **0.162** | — |
| via_rl REINFORCE+DIMR | **0.907** | 0.250 | 2.22× |
| via_rl v2 per-token | 0.713 | 0.104 | **3.53**× |

†wall-clock; via_*

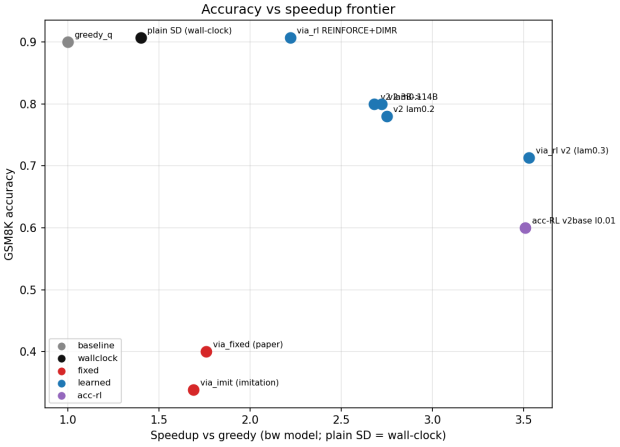
speedups are the overhead-free bandwidth model (not directly comparable).

# Latency: speedup by method




plain SD = measured wall-clock; via_* = overhead-free bandwidth model.

# The accuracy–speed frontier



# Why it matters

- We turn VIA-SD's **hand-tuned thresholds** into a **learned, per-token, context-aware** router.
- **Pareto-dominates** the fixed baseline; recovers *lossless* quality at far fewer full-model calls.
- **Free to deploy:** a tiny MLP on $q$-free features — no extra serving model, no $q$ at decision time.
- **Trajectory:** the gain *grows* with stronger drafters — where the field is heading.

*Stop hand-picking thresholds. Let the model schedule its own compute.*

# Future work

- **Spec-spec decoding:** make the drafter *itself* speculative — a recursive hierarchy; the gate routes across *levels*.
- **KnapSpec:** verification as a **knapsack** — a fixed budget of expensive calls spent on the highest-value tokens (value = correctness impact, weight = cost).
- **Correctness-floor RL:** minimize cost *subject to* an accuracy floor (avoids reward-hacking collapse).
- **Scale:** bigger drafters, more tasks, real-engine wall-clock (static cache / CUDA graphs).

# Learning to Route

Stop hand-picking thresholds — let the model schedule its own compute.

Thanks! questions?